

[illegible]

```

# Fit and Transform the documents
self.train_data = count_d.fit_transform(self.only_commit_text)
print("training data")
print(self.train_data)
self.terms = count_d.get_feature_names()
print(self.terms)
def lsi_formation(self, number_feature):
    # Create SVD object
    self.lsa = TruncatedSVD(n_components=self.f_number, n_iter=100,
random_state=42)

    # Fit SVD model on data
    self.lsa.fit_transform(self.train_data)

    # Get Singular values and Components
    Sigma = self.lsa.singular_values_
    print("^ matrix")
    print(Sigma)
    self.S_matrix = np.diag(Sigma)
    self.T_matrix = self.lsa.components_.T
    # print("T matrix")
    # print(self.T_matrix)
def manual_lsi(self):
    from sklearn.utils.extmath import randomized_svd

    U, Sigma, VT = randomized_svd(self.train_data,
                                n_components=self.f_number,
                                n_iter=5,
                                random_state=None)

    # Get Singular values and Components
    print("^ matrix")
    print(Sigma)
    self.S_matrix = np.diag(Sigma)
    self.T_matrix = np.transpose(VT)
    print("T matrix")
    print(self.T_matrix.shape)
    self.D_matrix = U
    print("D matrix")
    print(self.D_matrix.shape)
def manual_component_mapping(self):
    for index, component in enumerate(np.transpose(self.T_matrix)):
        # print("component for index", index)
        # print(component)
        zipped = zip(self.terms, component)
        dic_zip = dict(zip(self.terms, component))
        keys = list(dic_zip.keys())
        self.topic_weight.append((self.terms, dic_zip))

    # tops_svds.append(summ)

def manual_dq_formation(self, query):
    #TODO-  $DQ = Xq'TS^{-1}$ 
    print(query)
    Xq=[]
    for term_weight in self.topic_weight:
        terms, weights = term_weight
        row = []
        for term in terms:
            if term in query:
                row.append(weights[term])
            else:

```

```

        row.append(0)
        Xq.append(row)
    print("X-Q")
    xq = np.array(Xq)
    print(xq.shape)
    t_formed = np.transpose(xq)
    comparison = np.transpose(t_formed@self.S_matrix)
    concept_vectors = []
    for vec in comparison:
        concept_vectors.append(sum(vec))
    concept = concept_vectors.index(max(concept_vectors))
    # print("Comparison")
    # print(comparison)
    xx = np.transpose(xq)@self.S_matrix@np.transpose(self.D_matrix)
    self.Xihat = xx
    # print("xx shape", xx.shape)
    # print("COmparison")
    self.D_q = np.transpose(Xq)@self.T_matrix@np.linalg.inv(self.S_matrix)
    # print("D_q matrix")
    # print(self.D_q)
    # self.Xihat = np.transpose(Xq)@self.S_matrix@np.transpose(self.D_matrix)
    # self.D_q = np.transpose(xx)@self.T_matrix@np.linalg.inv(self.S_matrix)
    # print("D-q")
    # print(self.D_q.shape)

    s_inverse = np.linalg.inv(self.S_matrix)
    print("s_inverse", s_inverse.shape)
    # print(s_inverse)

    self.D_q = np.transpose(np.transpose(xx)@self.T_matrix@s_inverse)
    print("Centroid of Dq")
    self.centroids = np.flip(self.D_q[concept])#self.D_q[concept]#

def assumption_search(self, threshold):
    matched = []
    # for i,val in enumerate(self.centroids):
    #     if(val>0 and val<=threshold):
    #         matched.append(i)
    # print("found",index)
    # print(matched)
    vec = sum(self.centroids)
    dd = np.transpose(self.Xihat)@self.Xihat
    for i,d in enumerate(dd):
        dv = sum(d)
        result = abs(spatial.distance.cosine(dv, vec))
        if(result>0 and result<=threshold):
            matched.append(i)
    # print(matched)
    commit_ids = self.associate_commits_to_indices(matched)
    return commit_ids

def search_direct_components(self, features,path, data_path):
    ''' This methods use LSI approach for associating code components with issues'''

    feature_commits = []
    t_p = []
    t_r = []
    t_fl = []
    for feature in features:
        print("----- Feature Id-----:", feature[0])
        self.manual_dq_formation(self.preprocessing(feature[1]))
        ids = self.assumption_search(threshold=0.050)
        # print(ids)

```

```
#Main Method as the starting point of search
def direct_lsi_components_mapp(issue, commits, test_path, data_path):
    issu_content = Preprocess(issue, "")
    issu_content.readCsv()
    commit_content = Preprocess(commits, "")
    commit_content.readCodeCsv()
    # commit_content.filterCommitsByProgramFile()
    #print("commits", len(commit_content.content))
    lsi_experiment = lsi_model.LSI(commit_content.content, len(issu_content.content))
    # lsi_experiment.only_commit_text = documents_list
    lsi_experiment.prepare_commit_for_training()
    lsi_experiment.training_data()
    lsi_experiment.manual_lsi()
    lsi_experiment.manual_component_mapping()
    lsi_experiment.search_direct_components(issu_content.content, test_path, data_path)
```